

Option 2 of the Course Project:

Combinatorial Optimization - The Traveling Salesperson Problem (TSP)



Project Option 2: Combinatorial Optimization – The Traveling Salesperson Problem (TSP)

Title: Solving the Traveling Salesperson Problem: Heuristics and Approximation Algorithms



Project Description

The Traveling Salesperson Problem (TSP) is a cornerstone of combinatorics and optimization. Students will explore TSP, why it's hard to solve exactly, and implement a **heuristic or approximation algorithm** to find a solution.



Focus Concepts

- Combinatorics (permutations, factorial growth)
 - Graph theory (weighted complete graphs)
 - Greedy and heuristic algorithms
 - Approximation methods (Nearest Neighbor, Minimum Spanning Tree-based, 2-opt)
-



Written Report Requirements

- Define TSP and its mathematical formulation (graph, cost matrix)
- Explain why it's **NP-hard** and what that means
- Describe and analyze one or more algorithms, such as:
 - **Nearest Neighbor Heuristic**
 - **MST Approximation Algorithm**
 - **2-opt or 3-opt Improvement**

- Include:
 - Pseudocode of the algorithm
 - Full trace of the algorithm on a small example (e.g., 6 cities)
 - Time complexity discussion
 - Optional but encouraged: Discuss real-world applications (delivery routes, circuit design, genome sequencing)
-



Code Submission

- Implement the chosen TSP heuristic in Python, JavaScript, or another language
 - Accepts a list of coordinates or a distance matrix as input
 - Outputs the tour and total cost
 - Include test cases and output screenshot or logs
 - Encourage visualization (e.g., plotting the cities and tour path with matplotlib or browser-based canvas)
-



Video Presentation

- 5–8 minutes walking through:
 - Explanation of the TSP
 - How your algorithm works (slide and/or code demo)
 - Visualization of your solution on a sample graph
 - Discussion of efficiency and limitations
 - What you found challenging or interesting

You're welcome to use the following sample list of cities and coordinates in order to form a distance matrix (just use the classic Euclidean distance formula), and get started on the project:

City List and Coordinates

City	Label	Coordinates (x, y)
Central	A	(0, 0)
Northville	B	(2, 8)
Eastburg	C	(6, 3)

City	Label	Coordinates (x, y)
Southtown	D	(1, -5)
Westend	E	(-4, 3)
Lakeside	F	(-3, -4)

 Grading Rubric (Total: 100 points)

Category	Points	Criteria
1. Problem Understanding	15	- Clearly explains the TSP, including why it is NP-hard (5 pts) - Accurately defines problem using a graph or distance matrix (5 pts) - Correctly describes the chosen algorithm/heuristic (5 pts)
2. Algorithm Design & Pseudocode	15	- Presents clear, correct pseudocode for the chosen algorithm (8 pts) - Explains the logic and reasoning behind the algorithmic steps (7 pts)
3. Code Implementation	20	- Working code provided that correctly performs TSP heuristic (10 pts) - Code is well-commented and organized (5 pts) - At least one example is tested with outputs shown (5 pts)
4. Visualization	10	- Includes accurate and well-labeled plot or diagram of TSP route (e.g., matplotlib or draw.io) (5 pts) - Enhances understanding of the route and algorithm (5 pts)
5. Mathematical Depth	10	- Calculates total route cost correctly (3 pts) - Discusses algorithm's time complexity and limitations (5 pts) - Optional: compares results with brute-force or another heuristic (2 pts extra credit)
6. Written Report	10	- Organized, complete, and clearly written (4 pts) - Contains all required components (problem, pseudocode, analysis, reflection) (6 pts)
7. Video Presentation	15	- Clearly explains the problem, solution process, and code (7 pts) - Uses visuals (slides, screencast, diagrams) effectively (5 pts) - Stays within time limit (3 pts)
8. Engagement & Effort	5	- Shows original effort, curiosity, or attention to detail - Attempts to connect math to real-world ideas or challenges